

Course Name: Advanced System Analysis and Software
Design

Course Code: ENSF 480

Assignment Number: Lab 3

Lab Section: B01

Submission Date: 27/09/2026

Completed by:

Aayush Karthikeyan (UCID: 30189743)

Sarvesh Vettrivelan (UCID: 30242015)

Exercise A: Multiple Inheritance in C++

FILE: main.cpp

```
/*
 * File Name: main.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#include "graphicsWorld.h"
#include "curveCut.h"
#include <cstring>
#include <iostream>

int main(int argc, char* argv[]) {
    // Run the terminating error case separately from the normal
    demonstration.
    if (argc == 2 && std::strcmp(argv[1], "--invalid-radius") == 0) {
        CurveCut invalid(0, 0, 10, 12, 11, "INVALID CUT");
        std::cerr << "Error: invalid radius was accepted.\n";
        return 2;
    }
    GraphicsWorld graphic;
    graphic.run();
    return 0;
}
```

FILE: graphicsWorld.h

```
/*
 * File Name: graphicsWorld.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#ifndef GRAPHICSWORLD_H
#define GRAPHICSWORLD_H
```

```

class GraphicsWorld {
public:
    void run();
};

#endif

```

FILE: graphicsWorld.cpp

```

/*
 * File Name: graphicsWorld.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#include "graphicsWorld.h"
#include "square.h"
#include "rectangle.h"
#include "circle.h"
#include "curveCut.h"
#include <cassert>
#include <cstring>
#include <iostream>
using namespace std;

void GraphicsWorld::run() {
    cout << "Lab 3 by Sarvesh Vettrivelan and Aayush Karthikeyan" << endl;

    cout << "\nTesting functions in class Square:" << endl;
    Square s(5, 7, 12, "SQUARE - S");
    s.display();

    cout << "\nTesting functions in class Rectangle:" << endl;
    Rectangle a(5, 7, 12, 15, "RECTANGLE A");
    a.display();

    cout << "\nTesting functions in class Circle:" << endl;
    Circle c(3, 5, 9, "CIRCLE C");
    c.display();
    cout << "The area of " << c.getName() << " is: " << c.area() << endl;
    cout << "The perimeter of " << c.getName() << " is: " << c.perimeter()
<< endl;
    double d = a.distance(c);
    cout << "The distance between rectangle a and circle c is: " << d <<
endl;

    cout << "\nTesting functions in class CurveCut:" << endl;
    CurveCut rc(6, 5, 10, 12, 9, "CurveCut rc");
    rc.display();

```

```

    cout << "The area of " << rc.getName() << " is: " << rc.area() <<
endl;
    cout << "The perimeter of " << rc.getName() << " is: " <<
rc.perimeter() << endl;
    d = rc.distance(c);
    cout << "The distance between rc and c is: " << d << endl;

    cout << "\nTesting an array of Shape pointers:" << endl;
    Shape* sh[4];
    sh[0] = &s;
    sh[1] = &a;
    sh[2] = &c;
    sh[3] = &rc;
    for (int i = 0; i < 4; i++) {
        sh[i]->display();
        cout << "The area of " << sh[i]->getName() << " is: " <<
sh[i]->area() << endl;
        cout << "The perimeter of " << sh[i]->getName()
            << " is: " << sh[i]->perimeter() << endl << endl;
    }

    cout << "Testing copy constructor in class CurveCut:" << endl;
    CurveCut cc = rc;
    cc.display();
    assert(cc.getName() != rc.getName());
    assert(strcmp(cc.getName(), rc.getName()) == 0);

    cout << "\nTesting assignment operator in class CurveCut:" << endl;
    CurveCut cc2(2, 5, 100, 12, 9, "CurveCut cc2");
    cc2.display();
    cc2 = cc;
    cc2.display();
    assert(cc2.getName() != cc.getName());
    assert(strcmp(cc2.getName(), cc.getName()) == 0);

    cout << "\nProgram terminated successfully." << endl;
}

```

FILE: point.h

```

/*
 * File Name: point.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#ifndef POINT_H
#define POINT_H

class Point {

```

```

private:
    double ptX;
    double ptY;
    int id;
    static int count;
    static int nextId;
public:
    Point(double x, double y);
    Point(const Point& source);
    ~Point();
    Point& operator=(const Point& rhs);
    void display() const;
    void setX(double x);
    void setY(double y);
    double getX() const;
    double getY() const;
    int getId() const;
    static int counter();
    double distance(const Point& other) const;
    static double distance(const Point& first, const Point& second);
};
#endif

```

FILE: point.cpp

```

/*
 * File Name: point.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#include "point.h"
#include <cmath>
#include <iomanip>
#include <iostream>
using namespace std;

int Point::count = 0;
int Point::nextId = 1001;

Point::Point(double x, double y) : ptX(x), ptY(y), id(nextId++) {
    count++;
}

Point::Point(const Point& source)
    : ptX(source.ptX), ptY(source.ptY), id(nextId++) {
    count++;
}

Point::~~Point() { count--; }

Point& Point::operator=(const Point& rhs) {

```

```

        if (this != &rhs) {
            ptX = rhs.ptX;
            ptY = rhs.ptY;
        }
        // Assignment changes coordinates, but keeps this point's identity.
        return *this;
    }

    void Point::setX(double x) { ptX = x; }
    void Point::setY(double y) { ptY = y; }
    double Point::getX() const { return ptX; }
    double Point::getY() const { return ptY; }
    int Point::getId() const { return id; }
    int Point::counter() { return count; }

    double Point::distance(const Point& first, const Point& second) {
        double dx = first.ptX - second.ptX;
        double dy = first.ptY - second.ptY;
        return sqrt(dx * dx + dy * dy);
    }

    double Point::distance(const Point& other) const {
        return distance(*this, other);
    }

    void Point::display() const {
        ios::fmtflags oldFlags = cout.flags();
        streamsize oldPrecision = cout.precision();
        cout << fixed << setprecision(2);
        cout << "X-coordinate: " << ptX << '\n';
        cout << "Y-coordinate: " << ptY << '\n';
        cout.flags(oldFlags);
        cout.precision(oldPrecision);
    }
}

```

FILE: shape.h

```

/*
 * File Name: shape.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#ifndef SHAPE_H
#define SHAPE_H
#include "point.h"

class Shape {
private:
    Point origin;
    char* shapeName;
public:
    Shape(double x, double y, const char* name);

```

```

    Shape(const Shape& source);
    virtual ~Shape();
    Shape& operator=(const Shape& rhs);
    virtual double area() const = 0;
    virtual double perimeter() const = 0;
    virtual void display() const;
    const Point& getOrigin() const;
    const char* getName() const;
    double distance(const Shape& other) const;
    static double distance(const Shape& first, const Shape& second);
    void move(double dx, double dy);
};
#endif

```

FILE: shape.cpp

```

/*
 * File Name: shape.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#include "shape.h"
#include <cstring>
#include <iostream>
using namespace std;

static char* copyString(const char* s) {
    if (s == nullptr)
        s = "";
    char* p = new char[strlen(s) + 1];
    strcpy(p, s);
    return p;
}

Shape::Shape(double x, double y, const char* name)
    : origin(x, y), shapeName(copyString(name)) {}

Shape::Shape(const Shape& source)
    : origin(source.origin), shapeName(copyString(source.shapeName)) {}

Shape::~Shape() { delete[] shapeName; }

Shape& Shape::operator=(const Shape& rhs) {
    if (this != &rhs) {
        char* newName = copyString(rhs.shapeName);
        delete[] shapeName;
        shapeName = newName;
        origin = rhs.origin;
    }
    return *this;
}

void Shape::display() const {
    cout << "Shape Name: " << shapeName << '\n';
}

```

```

        origin.display();
    }
    const Point& Shape::getOrigin() const { return origin; }
    const char* Shape::getName() const { return shapeName; }
    double Shape::distance(const Shape& first, const Shape& second) {
        return Point::distance(first.origin, second.origin);
    }
    double Shape::distance(const Shape& other) const {
        return Point::distance(origin, other.origin);
    }
    void Shape::move(double dx, double dy) {
        origin.setX(origin.getX() + dx);
        origin.setY(origin.getY() + dy);
    }
}

```

FILE: square.h

```

/*
 * File Name: square.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#ifndef SQUARE_H
#define SQUARE_H
#include "shape.h"

// Square and Circle share one Shape subobject inside CurveCut.
class Square : virtual public Shape {
private:
    double side_a;
public:
    Square(double x, double y, double sideA, const char* name);
    double get_side_a() const;
    virtual void set_side_a(double sideA);
    double area() const override;
    double perimeter() const override;
    void display() const override;
};
#endif

```

FILE: square.cpp

```

/*
 * File Name: square.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan

```

```

* Submission Date: Sept. 25, 2026
*/

#include "square.h"
#include <cmath>
#include <cstdlib>
#include <iostream>
using namespace std;

Square::Square(double x, double y, double sideA, const char* name)
    : Shape(x, y, name), side_a(sideA) {
    set_side_a(sideA);
}

double Square::get_side_a() const { return side_a; }
void Square::set_side_a(double sideA) {
    if (!isfinite(sideA) || sideA < 0) {
        cerr << "Error: side a must be a finite, non-negative value.\n";
        exit(EXIT_FAILURE);
    }
    side_a = sideA;
}

double Square::area() const { return side_a * side_a; }
double Square::perimeter() const { return 4 * side_a; }
void Square::display() const {
    cout << "Square Name: " << getName() << '\n';
    getOrigin().display();
    cout << "Side a: " << side_a << '\n';
    cout << "Area: " << area() << '\n';
    cout << "Perimeter: " << perimeter() << '\n';
}

```

FILE: rectangle.h

```

/*
* File Name: rectangle.h
* Assignment: Lab 3 Exercise A
* Completed By:
*   - Sarvesh Vettrivelan
*   - Aayush Karthikeyan
* Submission Date: Sept. 25, 2026
*/

#ifndef RECTANGLE_H
#define RECTANGLE_H
#include "square.h"

class Rectangle : public Square {
private:
    double side_b;
public:
    Rectangle(double x, double y, double sideA, double sideB, const char*
name);
    double get_side_b() const;
    virtual void set_side_b(double sideB);

```

```

    double area() const override;
    double perimeter() const override;
    void display() const override;
};
#endif

```

FILE: rectangle.cpp

```

/*
 * File Name: rectangle.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#include "rectangle.h"
#include <cmath>
#include <cstdlib>
#include <iostream>
using namespace std;

Rectangle::Rectangle(double x, double y, double sideA, double sideB,
                    const char* name)
    : Shape(x, y, name), Square(x, y, sideA, name), side_b(sideB) {
    set_side_b(sideB);
}

double Rectangle::get_side_b() const { return side_b; }
void Rectangle::set_side_b(double sideB) {
    if (!isfinite(sideB) || sideB < 0) {
        cerr << "Error: side b must be a finite, non-negative value.\n";
        exit(EXIT_FAILURE);
    }
    side_b = sideB;
}

double Rectangle::area() const { return get_side_a() * side_b; }
double Rectangle::perimeter() const { return 2 * (get_side_a() + side_b); }
}

void Rectangle::display() const {
    cout << "Rectangle Name: " << getName() << '\n';
    getOrigin().display();
    cout << "Side a: " << get_side_a() << '\n';
    cout << "Side b: " << side_b << '\n';
    cout << "Area: " << area() << '\n';
    cout << "Perimeter: " << perimeter() << '\n';
}

```

FILE: circle.h

```

/*

```

```

* File Name: circle.h
* Assignment: Lab 3 Exercise A
* Completed By:
*   - Sarvesh Vettrivelan
*   - Aayush Karthikeyan
* Submission Date: Sept. 25, 2026
*/

#ifndef CIRCLE_H
#define CIRCLE_H
#include "shape.h"

class Circle : virtual public Shape {
private:
    double radius;
public:
    Circle(double x, double y, double r, const char* name);
    double get_radius() const;
    virtual void set_radius(double r);
    double area() const override;
    double perimeter() const override;
    void display() const override;
};
#endif

```

FILE: circle.cpp

```

/*
* File Name: circle.cpp
* Assignment: Lab 3 Exercise A
* Completed By:
*   - Sarvesh Vettrivelan
*   - Aayush Karthikeyan
* Submission Date: Sept. 25, 2026
*/

#include "circle.h"
#include <cmath>
#include <cstdlib>
#include <iostream>
using namespace std;

Circle::Circle(double x, double y, double r, const char* name)
    : Shape(x, y, name), radius(r) {
    set_radius(r);
}

double Circle::get_radius() const { return radius; }
void Circle::set_radius(double r) {
    if (!isfinite(r) || r < 0) {
        cerr << "Error: radius must be a finite, non-negative value.\n";
        exit(EXIT_FAILURE);
    }
    radius = r;
}

```

```

}
double Circle::area() const { return acos(-1.0) * radius * radius; }
double Circle::perimeter() const { return 2 * acos(-1.0) * radius; }
void Circle::display() const {
    cout << "Circle Name: " << getName() << '\n';
    getOrigin().display();
    cout << "Radius: " << radius << '\n';
    cout << "Area: " << area() << '\n';
    cout << "Perimeter: " << perimeter() << '\n';
}

```

FILE: curveCut.h

```

/*
 * File Name: curveCut.h
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#ifndef CURVECUT_H
#define CURVECUT_H
#include "rectangle.h"
#include "circle.h"

class CurveCut : public Rectangle, public Circle {
private:
    void check_dimensions(double width, double length, double r) const;
public:
    CurveCut(double x, double y, double width, double length, double r,
              const char* name);
    CurveCut(const CurveCut& source);
    CurveCut& operator=(const CurveCut& rhs);
    void set_side_a(double width) override;
    void set_side_b(double length) override;
    void set_radius(double r) override;
    double area() const override;
    double perimeter() const override;
    void display() const override;
};
#endif

```

FILE: curveCut.cpp

```

/*
 * File Name: curveCut.cpp
 * Assignment: Lab 3 Exercise A
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026

```

```

*/

#include "curveCut.h"
#include <cmath>
#include <cstdlib>
#include <iostream>
using namespace std;

CurveCut::CurveCut(double x, double y, double width, double length, double
r,
                    const char* name)
    : Shape(x, y, name), Rectangle(x, y, width, length, name),
      Circle(x, y, r, name) {
    check_dimensions(width, length, r);
}

CurveCut::CurveCut(const CurveCut& source)
    : Shape(source), Rectangle(source), Circle(source) {}

CurveCut& CurveCut::operator=(const CurveCut& rhs) {
    if (this != &rhs) {
        // Copy the shared Shape once. The source dimensions are already
valid.
        Shape::operator=(rhs);
        Rectangle::set_side_a(rhs.get_side_a());
        Rectangle::set_side_b(rhs.get_side_b());
        Circle::set_radius(rhs.get_radius());
    }
    return *this;
}

void CurveCut::check_dimensions(double width, double length, double r)
const {
    if (!isfinite(width) || !isfinite(length) || !isfinite(r) ||
        width < 0 || length < 0 || r < 0 || r > width || r > length) {
        cerr << "Error: CurveCut needs non-negative dimensions and a
radius "
                << "no greater than its smaller side.\n";
        exit(EXIT_FAILURE);
    }
}

void CurveCut::set_side_a(double width) {
    check_dimensions(width, get_side_b(), get_radius());
    Rectangle::set_side_a(width);
}

void CurveCut::set_side_b(double length) {
    check_dimensions(get_side_a(), length, get_radius());
    Rectangle::set_side_b(length);
}

void CurveCut::set_radius(double r) {
    check_dimensions(get_side_a(), get_side_b(), r);
    Circle::set_radius(r);
}

double CurveCut::area() const {
    return Rectangle::area() - Circle::area() / 4;
}

```

```

double CurveCut::perimeter() const {
    // Two straight sections of length r are replaced by a quarter-circle
    arc.
    return Rectangle::perimeter() - 2 * get_radius() + Circle::perimeter()
    / 4;
}
void CurveCut::display() const {
    cout << "CurveCut Name: " << getName() << '\n';
    getOrigin().display();
    cout << "Width: " << get_side_a() << '\n';
    cout << "Length: " << get_side_b() << '\n';
    cout << "Radius of the cut: " << get_radius() << '\n';
}

```

Exercise A: Program Output

Lab 3 by Sarvesh Vettrivelan and Aayush Karthikeyan

Testing functions in class Square:

Square Name: SQUARE - S
 X-coordinate: 5.00
 Y-coordinate: 7.00
 Side a: 12
 Area: 144
 Perimeter: 48

Testing functions in class Rectangle:

Rectangle Name: RECTANGLE A
 X-coordinate: 5.00
 Y-coordinate: 7.00
 Side a: 12
 Side b: 15
 Area: 180
 Perimeter: 54

Testing functions in class Circle:

Circle Name: CIRCLE C
 X-coordinate: 3.00
 Y-coordinate: 5.00
 Radius: 9
 Area: 254.469
 Perimeter: 56.5487
 The area of CIRCLE C is: 254.469
 The perimeter of CIRCLE C is: 56.5487
 The distance between rectangle a and circle c is: 2.82843

Testing functions in class CurveCut:

CurveCut Name: CurveCut rc
 X-coordinate: 6.00
 Y-coordinate: 5.00
 Width: 10

Length: 12
Radius of the cut: 9
The area of CurveCut rc is: 56.3827
The perimeter of CurveCut rc is: 40.1372
The distance between rc and c is: 3

Testing an array of Shape pointers:
Square Name: SQUARE - S
X-coordinate: 5.00
Y-coordinate: 7.00
Side a: 12
Area: 144
Perimeter: 48
The area of SQUARE - S is: 144
The perimeter of SQUARE - S is: 48

Rectangle Name: RECTANGLE A
X-coordinate: 5.00
Y-coordinate: 7.00
Side a: 12
Side b: 15
Area: 180
Perimeter: 54
The area of RECTANGLE A is: 180
The perimeter of RECTANGLE A is: 54

Circle Name: CIRCLE C
X-coordinate: 3.00
Y-coordinate: 5.00
Radius: 9
Area: 254.469
Perimeter: 56.5487
The area of CIRCLE C is: 254.469
The perimeter of CIRCLE C is: 56.5487

CurveCut Name: CurveCut rc
X-coordinate: 6.00
Y-coordinate: 5.00
Width: 10
Length: 12
Radius of the cut: 9
The area of CurveCut rc is: 56.3827
The perimeter of CurveCut rc is: 40.1372

Testing copy constructor in class CurveCut:
CurveCut Name: CurveCut rc
X-coordinate: 6.00
Y-coordinate: 5.00
Width: 10
Length: 12
Radius of the cut: 9

Testing assignment operator in class CurveCut:

CurveCut Name: CurveCut cc2

X-coordinate: 2.00

Y-coordinate: 5.00

Width: 100

Length: 12

Radius of the cut: 9

CurveCut Name: CurveCut rc

X-coordinate: 6.00

Y-coordinate: 5.00

Width: 10

Length: 12

Radius of the cut: 9

Program terminated successfully.

Exercise B: Templates in C++

FILE: iterator.cpp

```
/*
 * File Name: iterator.cpp
 * Assignment: Lab 3 Exercise B
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

// Based on the Vector and VectIter starter code supplied for Lab 3.
#include <cassert>
#include <cstring>
#include <iostream>
#include "mystring2.h"
using namespace std;

template <class T>
class Vector {
public:
    class VectIter {
    private:
        Vector<T>* v;
        int index;
    public:
        explicit VectIter(Vector<T>& x);
        T operator++();
        T operator++(int);
        T operator--();
        T operator--(int);
        T operator*() const;
    };

    explicit Vector(int sz);
    Vector(const Vector<T>& source);
    Vector<T>& operator=(const Vector<T>& rhs);
```

```

~Vector();

T& operator[](int i);
const T& operator[](int i) const;
void ascending_sort();
private:
    T* array;
    int size;
    void swap(T& a, T& b);
};

template <class T>
Vector<T>::Vector(int sz) : array(nullptr), size(sz) {
    assert(sz >= 0);
    array = new T[sz]();
}

template <class T>
Vector<T>::Vector(const Vector<T>& source)
    : array(new T[source.size]), size(source.size) {
    try {
        for (int i = 0; i < size; i++)
            array[i] = source.array[i];
    } catch (...) {
        delete[] array;
        throw;
    }
}

template <class T>
Vector<T>& Vector<T>::operator=(const Vector<T>& rhs) {
    if (this != &rhs) {
        Vector<T> copy(rhs);
        T* oldArray = array;
        array = copy.array;
        copy.array = oldArray;
        int oldSize = size;
        size = copy.size;
        copy.size = oldSize;
    }
    return *this;
}

```

```

}

template <class T>
Vector<T>::~~Vector() { delete[] array; }

template <class T>
T& Vector<T>::operator[](int i) {
    assert(i >= 0 && i < size);
    return array[i];
}

template <class T>
const T& Vector<T>::operator[](int i) const {
    assert(i >= 0 && i < size);
    return array[i];
}

template <class T>
void Vector<T>::swap(T& a, T& b) {
    T tmp = a;
    a = b;
    b = tmp;
}

template <class T>
void Vector<T>::ascending_sort() {
    for (int i = 0; i < size - 1; i++)
        for (int j = i + 1; j < size; j++)
            if (array[i] > array[j])
                swap(array[i], array[j]);
}

// C strings need content comparison; comparing pointers sorts addresses.
template <>
void Vector<const char*>::ascending_sort() {
    for (int i = 0; i < size - 1; i++)
        for (int j = i + 1; j < size; j++)
            if (strcmp(array[i], array[j]) > 0)
                swap(array[i], array[j]);
}

```

```

template <>
void Vector<char*>::ascending_sort() {
    for (int i = 0; i < size - 1; i++)
        for (int j = i + 1; j < size; j++)
            if (strcmp(array[i], array[j]) > 0)
                swap(array[i], array[j]);
}

template <class T>
Vector<T>::VectIter::VectIter(Vector<T>& x) : v(&x), index(0) {
    // A circular iterator needs at least one element.
    assert(v->size > 0);
}

template <class T>
T Vector<T>::VectIter::operator*() const { return v->array[index]; }

template <class T>
T Vector<T>::VectIter::operator++() {
    index++;
    if (index == v->size)
        index = 0;
    return v->array[index];
}

template <class T>
T Vector<T>::VectIter::operator++(int) {
    T value = v->array[index];
    ++(*this);
    return value;
}

template <class T>
T Vector<T>::VectIter::operator--() {
    index--;
    if (index < 0)
        index = v->size - 1;
    return v->array[index];
}

```

```

template <class T>
T Vector<T>::VectIter::operator--(int) {
    T value = v->array[index];
    --(*this);
    return value;
}

int main() {
    Vector<int> x(3);
    x[0] = 999;
    x[1] = -77;
    x[2] = 88;
    Vector<int>::VectIter iter(x);

    cout << "The first element of vector x contains: " << *iter << endl;

#ifdef 1
    cout << "\nTesting an <int> Vector:" << endl;
    cout << "\nTesting sort and postfix ++:" << endl;
    x.sort();
    assert(x[0] == -77 && x[1] == 88 && x[2] == 999);
    for (int i = 0; i < 3; i++)
        cout << iter++ << endl;

    cout << "\nTesting prefix --:" << endl;
    for (int i = 0; i < 3; i++)
        cout << --iter << endl;

    cout << "\nTesting prefix ++:" << endl;
    for (int i = 0; i < 3; i++)
        cout << ++iter << endl;

    cout << "\nTesting postfix --:" << endl;
    for (int i = 0; i < 3; i++)
        cout << iter-- << endl;

    cout << "\nTesting a <Mystring> Vector:" << endl;
    Vector<Mystring> y(3);
    y[0] = "Bar";

```

```

y[1] = "Foo";
y[2] = "All";
Vector<Mystring>::VectIter iters(y);

cout << "\nTesting sort and postfix ++:" << endl;
y.ascending_sort();
assert(y[0] == "All" && y[1] == "Bar" && y[2] == "Foo");
for (int i = 0; i < 3; i++)
    cout << iters++ << endl;

cout << "\nTesting prefix --:" << endl;
for (int i = 0; i < 3; i++)
    cout << --iters << endl;

cout << "\nTesting prefix ++:" << endl;
for (int i = 0; i < 3; i++)
    cout << ++iters << endl;

cout << "\nTesting postfix --:" << endl;
for (int i = 0; i < 3; i++)
    cout << iters-- << endl;

cout << "\nTesting a <const char*> Vector:" << endl;
Vector<const char*> z(3);
z[0] = "Orange";
z[1] = "Pear";
z[2] = "Apple";
Vector<const char*>::VectIter iterchar(z);

cout << "\nTesting sort and postfix ++:" << endl;
z.ascending_sort();
assert(strcmp(z[0], "Apple") == 0);
assert(strcmp(z[1], "Orange") == 0);
assert(strcmp(z[2], "Pear") == 0);
for (int i = 0; i < 3; i++)
    cout << iterchar++ << endl;

cout << "\nTesting prefix --:" << endl;
for (int i = 0; i < 3; i++)
    cout << --iterchar << endl;

```

```

    cout << "\nTesting prefix ++:" << endl;
    for (int i = 0; i < 3; i++)
        cout << ++iterchar << endl;

    cout << "\nTesting postfix --:" << endl;
    for (int i = 0; i < 3; i++)
        cout << iterchar-- << endl;
#endif

    cout << "\nProgram terminated successfully." << endl;
    return 0;
}

```

FILE: mystring2.h

```

/*
 * File Name: mystring2.h
 * Assignment: Lab 3 Exercise B
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#ifndef MYSTRING_H
#define MYSTRING_H
#include <iostream>

class Mystring {
    friend std::ostream& operator<<(std::ostream& os, const Mystring& s);
private:
    int lengthM;
    char* charsM;
public:
    Mystring();
    explicit Mystring(int n);
    Mystring(const char* s);
    Mystring(const Mystring& source);
    ~Mystring();

```

```

    Mystring& operator=(const Mystring& rhs);
    int length() const;
    char get_char(int pos) const;
    const char* c_str() const;
    void set_char(int pos, char c);
    Mystring& append(const Mystring& other);
    void set_str(const char* s);
    bool operator<(const Mystring& rhs) const;
    bool operator>(const Mystring& rhs) const;
    bool operator==(const Mystring& rhs) const;
    bool operator!=(const Mystring& rhs) const;
    bool operator<=(const Mystring& rhs) const;
    bool operator>=(const Mystring& rhs) const;
};
#endif

```

FILE: mystring2.cpp

```

/*
 * File Name: mystring2.cpp
 * Assignment: Lab 3 Exercise B
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#include "mystring2.h"
#include <cassert>
#include <cstring>

Mystring::Mystring() : lengthM(0), charsM(new char[1]) {
    charsM[0] = '\0';
}

Mystring::Mystring(int n) : lengthM(0), charsM(nullptr) {
    assert(n > 0);
    charsM = new char[n];
    charsM[0] = '\0';
}

Mystring::Mystring(const char* s) : lengthM(0), charsM(nullptr) {

```

```

    assert(s != nullptr);
    lengthM = static_cast<int>(std::strlen(s));
    charsM = new char[lengthM + 1];
    std::strcpy(charsM, s);
}

Mystring::Mystring(const Mystring& source)
    : lengthM(source.lengthM), charsM(new char[source.lengthM + 1]) {
    std::strcpy(charsM, source.charsM);
}

Mystring::~~Mystring() { delete[] charsM; }

Mystring& Mystring::operator=(const Mystring& rhs) {
    if (this != &rhs)
        set_str(rhs.charsM);
    return *this;
}

int Mystring::length() const { return lengthM; }

char Mystring::get_char(int pos) const {
    assert(pos >= 0 && pos < lengthM);
    return charsM[pos];
}

const char* Mystring::c_str() const { return charsM; }

void Mystring::set_char(int pos, char c) {
    assert(pos >= 0 && pos < lengthM && c != '\\0');
    charsM[pos] = c;
}

Mystring& Mystring::append(const Mystring& other) {
    int newLength = lengthM + other.lengthM;
    char* tmp = new char[newLength + 1];
    std::strcpy(tmp, charsM);
    std::strcat(tmp, other.charsM);
    delete[] charsM;
    charsM = tmp;
    lengthM = newLength;
    return *this;
}

void Mystring::set_str(const char* s) {
    assert(s != nullptr);
    int newLength = static_cast<int>(std::strlen(s));
    // Copy before deleting so s may refer to part of this string.
    char* tmp = new char[newLength + 1];

```

```

        std::strcpy(tmp, s);
        delete[] charsM;
        charsM = tmp;
        lengthM = newLength;
    }

    bool Mystring::operator<(const Mystring& rhs) const {
        return std::strcmp(charsM, rhs.charsM) < 0;
    }

    bool Mystring::operator>(const Mystring& rhs) const { return rhs < *this; }

    bool Mystring::operator==(const Mystring& rhs) const {
        return std::strcmp(charsM, rhs.charsM) == 0;
    }

    bool Mystring::operator!=(const Mystring& rhs) const { return !(*this ==
rhs); }

    bool Mystring::operator<=(const Mystring& rhs) const { return !(rhs <
*this); }

    bool Mystring::operator>=(const Mystring& rhs) const { return !(*this <
rhs); }

    std::ostream& operator<<(std::ostream& os, const Mystring& s) {
        return os << s.charsM;
    }

```

Exercise B: Program Output

The first element of vector x contains: 999

Testing an <int> Vector:

Testing sort and postfix ++:

-77

88

999

Testing prefix --:

999

88

-77

Testing prefix ++:

88

999

-77

Testing postfix --:

-77

999

88

Testing a <Mystring> Vector:

Testing sort and postfix ++:

All

Bar

Foo

Testing prefix --:

Foo

Bar

All

Testing prefix ++:

Bar

Foo

All

Testing postfix --:

All

Foo

Bar

Testing a <const char*> Vector:

Testing sort and postfix ++:
Apple
Orange
Pear

Testing prefix --:
Pear
Orange
Apple

Testing prefix ++:
Orange
Pear
Apple

Testing postfix --:
Apple
Pear
Orange

Program terminated successfully.

Exercise C: Template Lookup Table

FILE: lookupTable.h

```
/*
 * File Name: lookupTable.h
 * Assignment: Lab 3 Exercise C
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

// Based on the linked lookup table supplied for ENSF 480 Lab 3.
#ifndef LOOKUPTABLE_H
#define LOOKUPTABLE_H

#include <cassert>
#include <cstring>
#include <iostream>
#include <utility>

// Keys need a strict ordering. C-string keys are compared by their
// contents.
template <class K>
bool lt_less(const K& a, const K& b) { return a < b; }

template <>
inline bool lt_less<const char*>(const char* const& a, const char* const&
b) {
    return std::strcmp(a, b) < 0;
}

template <>
inline bool lt_less<char*>(char* const& a, char* const& b) {
    return std::strcmp(a, b) < 0;
}

template <class K, class D> class LookupTable;
template <class K, class D>
std::ostream& operator<<(std::ostream& os, const LookupTable<K, D>& lt);
```

```

template <class K, class D>
struct Pair {
    Pair(const K& keyA, const D& datumA) : key(keyA), datum(datumA) {}
    K key;
    D datum;
};

template <class K, class D>
class LT_Node {
    friend class LookupTable<K, D>;
private:
    Pair<K, D> pairM;
    LT_Node<K, D>* nextM;
    LT_Node(const Pair<K, D>& pairA, LT_Node<K, D>* nextA)
        : pairM(pairA), nextM(nextA) {}
};

template <class K, class D>
class LookupTable {
    friend std::ostream& operator<< <K, D>(std::ostream& os,
                                           const LookupTable<K, D>& lt);
public:
    // As in the starter code, iterators use their table's shared cursor.
    class Iterator {
    private:
        LookupTable<K, D>* LT;
    public:
        Iterator() : LT(nullptr) {}
        Iterator(LookupTable<K, D>& x) : LT(&x) {}
        const D& operator*() const;
        // Keep the Exercise C starter behavior: return current data, then
        advance.
        const D& operator++();
        // The starter's postfix version advances first and requires a
        next node.
        const D& operator++(int);
        // The supplied interface uses !it to mean that the cursor IS
        valid.
        int operator!() const;
        void step_fwd();

```

```

};

LookupTable();
LookupTable(const LookupTable<K, D>& source);
LookupTable<K, D>& operator=(const LookupTable<K, D>& rhs);
~LookupTable();
LookupTable<K, D>& begin();
int size() const;
int cursor_ok() const;
const K& cursor_key() const;
const D& cursor_datum() const;
// Insert replaces an existing key's datum or adds a new pair in
order.
// Insert and remove always put the cursor in the off-list state.
void insert(const Pair<K, D>& pairA);
void remove(const K& keyA);
void find(const K& keyA);
void go_to_first();
void step_fwd();
void make_empty();
private:
    int sizeM;
    LT_Node<K, D>* headM;
    LT_Node<K, D>* cursorM;
    void destroy();
    void copy(const LookupTable<K, D>& source);
};

template <class K, class D>
LookupTable<K, D>::LookupTable() : sizeM(0), headM(nullptr),
cursorM(nullptr) {}

template <class K, class D>
LookupTable<K, D>::LookupTable(const LookupTable<K, D>& source)
    : sizeM(0), headM(nullptr), cursorM(nullptr) {
    copy(source);
}

template <class K, class D>

```

```

LookupTable<K, D>& LookupTable<K, D>::operator=(const LookupTable<K, D>&
rhs) {
    if (this != &rhs) {
        LookupTable<K, D> temp(rhs);
        std::swap(sizeM, temp.sizeM);
        std::swap(headM, temp.headM);
        std::swap(cursorM, temp.cursorM);
    }
    return *this;
}

template <class K, class D>
LookupTable<K, D>::~~LookupTable() { destroy(); }

template <class K, class D>
LookupTable<K, D>& LookupTable<K, D>::begin() {
    go_to_first();
    return *this;
}

template <class K, class D>
int LookupTable<K, D>::size() const { return sizeM; }

template <class K, class D>
int LookupTable<K, D>::cursor_ok() const { return cursorM != nullptr; }

template <class K, class D>
const K& LookupTable<K, D>::cursor_key() const {
    assert(cursor_ok());
    return cursorM->pairM.key;
}

template <class K, class D>
const D& LookupTable<K, D>::cursor_datum() const {
    assert(cursor_ok());
    return cursorM->pairM.datum;
}

template <class K, class D>
void LookupTable<K, D>::insert(const Pair<K, D>& pairA) {

```

```

    cursorM = nullptr;
    LT_Node<K, D>* before = nullptr;
    LT_Node<K, D>* after = headM;
    while (after != nullptr && lt_less(after->pairM.key, pairA.key)) {
        before = after;
        after = after->nextM;
    }
    // Neither key is less than the other, so they refer to the same
entry.
    if (after != nullptr && !lt_less(pairA.key, after->pairM.key)) {
        after->pairM.datum = pairA.datum;
        return;
    }
    LT_Node<K, D>* node = new LT_Node<K, D>(pairA, after);
    if (before == nullptr)
        headM = node;
    else
        before->nextM = node;
    sizeM++;
}

```

```

template <class K, class D>
void LookupTable<K, D>::remove(const K& keyA) {
    cursorM = nullptr;
    LT_Node<K, D>* before = nullptr;
    LT_Node<K, D>* node = headM;
    while (node != nullptr && lt_less(node->pairM.key, keyA)) {
        before = node;
        node = node->nextM;
    }
    if (node == nullptr || lt_less(keyA, node->pairM.key))
        return;
    if (before == nullptr)
        headM = node->nextM;
    else
        before->nextM = node->nextM;
    delete node;
    sizeM--;
}

```

```

template <class K, class D>
void LookupTable<K, D>::find(const K& keyA) {
    cursorM = headM;
    while (cursorM != nullptr && lt_less(cursorM->pairM.key, keyA))
        cursorM = cursorM->nextM;
    if (cursorM != nullptr && lt_less(keyA, cursorM->pairM.key))
        cursorM = nullptr;
}

template <class K, class D>
void LookupTable<K, D>::go_to_first() { cursorM = headM; }

template <class K, class D>
void LookupTable<K, D>::step_fwd() {
    assert(cursor_ok());
    cursorM = cursorM->nextM;
}

template <class K, class D>
void LookupTable<K, D>::make_empty() { destroy(); }

template <class K, class D>
void LookupTable<K, D>::destroy() {
    while (headM != nullptr) {
        LT_Node<K, D>* old = headM;
        headM = headM->nextM;
        delete old;
    }
    cursorM = nullptr;
    sizeM = 0;
}

template <class K, class D>
void LookupTable<K, D>::copy(const LookupTable<K, D>& source) {
    LT_Node<K, D>* tail = nullptr;
    try {
        for (LT_Node<K, D>* p = source.headM; p != nullptr; p = p->nextM)
        {
            LT_Node<K, D>* node = new LT_Node<K, D>(p->pairM, nullptr);
            if (tail == nullptr)

```

```

        headM = node;
    else
        tail->nextM = node;
        tail = node;
        sizeM++;
        if (p == source.cursorM)
            cursorM = node;
    }
} catch (...) {
    destroy();
    throw;
}
}

template <class K, class D>
std::ostream& operator<<(std::ostream& os, const LookupTable<K, D>& lt) {
    if (lt.cursor_ok())
        os << lt.cursor_key() << " " << lt.cursor_datum();
    else
        os << "Not Found.";
    return os;
}

template <class K, class D>
const D& LookupTable<K, D>::Iterator::operator*() const {
    assert(LT != nullptr && LT->cursor_ok());
    return LT->cursor_datum();
}

template <class K, class D>
const D& LookupTable<K, D>::Iterator::operator++() {
    assert(LT != nullptr && LT->cursor_ok());
    const D& value = LT->cursor_datum();
    LT->step_fwd();
    return value;
}

template <class K, class D>
const D& LookupTable<K, D>::Iterator::operator++(int) {

```

```

        assert(LT != nullptr && LT->cursorM != nullptr && LT->cursorM->nextM
!= nullptr);
        LT->step_fwd();
        return LT->cursor_datum();
    }

template <class K, class D>
int LookupTable<K, D>::Iterator::operator!() const {
    return LT != nullptr && LT->cursor_ok();
}

template <class K, class D>
void LookupTable<K, D>::Iterator::step_fwd() {
    assert(LT != nullptr && LT->cursor_ok());
    LT->step_fwd();
}

#endif

```

FILE: mainLab3ExC.cpp

```

/*
 * File Name: mainLab3ExC.cpp
 * Assignment: Lab 3 Exercise C
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#include <cassert>
#include <cstring>
#include <iostream>
#include "lookupTable.h"
#include "customer.h"
using namespace std;

template <class K, class D>
void print(LookupTable<K, D>& lt);

```

```

template <class K, class D>
void try_to_find(LookupTable<K, D>& lt, const K& key);

void test_Customer();
void test_String();
void test_integer();

int main() {
    test_Customer();
    test_String();
    test_integer();

    cout << "\nProgram terminated successfully." << endl;
    return 0;
}

template <class K, class D>
void print(LookupTable<K, D>& lt) {
    if (lt.size() == 0)
        cout << "  Table is EMPTY." << endl;
    for (lt.go_to_first(); lt.cursor_ok(); lt.step_fwd()) {
        cout << lt << endl;
    }
}

template <class K, class D>
void try_to_find(LookupTable<K, D>& lt, const K& key) {
    lt.find(key);
    if (lt.cursor_ok())
        cout << "Found key: " << lt << endl;
    else
        cout << "Sorry, I couldn't find key: " << key << " in the table."
<< endl;
}

void test_Customer() {
    cout << "\nCreating and testing LookupTable<int, Customer>..." <<
endl;
    LookupTable<int, Customer> lt;

```

```

Customer a("Joe", "Morrison", "11 St. Calgary", "(403)-1111-123333");
Customer b("Jack", "Lewis", "12 St. Calgary", "(403)-1111-123334");
Customer c("Tim", "Hardy", "13 St. Calgary", "(403)-1111-123335");
lt.insert(Pair<int, Customer>(8002, a));
lt.insert(Pair<int, Customer>(8004, c));
lt.insert(Pair<int, Customer>(8001, b));
assert(lt.size() == 3);
lt.remove(8004);
assert(lt.size() == 2);
cout << "\nPrinting table after inserting 3 new keys and 1 removal:"
<< endl;
print(lt);

cout << "\nLet's look up some names:" << endl;
try_to_find(lt, 8001);
try_to_find(lt, 8000);

cout << "\nTesting the iterator:" << endl;
LookupTable<int, Customer>::Iterator it = lt.begin();
cout << "The first node contains: " << *it << endl;
while (!it) {
    cout << ++it << endl;
}
assert(!lt.cursor_ok());

// Test the copy constructor with the cursor at the second node.
lt.go_to_first();
lt.step_fwd();
LookupTable<int, Customer> clt(lt);
assert(clt.size() == 2);
assert(strcmp(clt.cursor_datum().getFname(), "Joe") == 0);
cout << "\nTest copying: keys should be 8001 and 8002" << endl;
print(clt);
lt.remove(8002);
assert(clt.size() == 2);

clt = lt;
assert(clt.size() == 1);
cout << "\nTest assignment operator: key should be 8001" << endl;
print(clt);

```

```

        lt.make_empty();
        assert(lt.size() == 0);
        cout << "\nPrinting table for the last time: table should be empty" <<
endl;
        print(lt);
        cout << "*** Finished tests on LookupTable<int, Customer> ***" <<
endl;
    }

void test_String() {
    cout << "\nCreating and testing LookupTable<int, Mystring>..." <<
endl;
    LookupTable<int, Mystring> lt;

    Mystring a("I am an ENSF 480 student.");
    Mystring b("C++ is a powerful language for engineers but it's not
easy.");
    Mystring c("Fall 2026");
    lt.insert(Pair<int, Mystring>(8002, a));
    lt.insert(Pair<int, Mystring>(8001, b));
    lt.insert(Pair<int, Mystring>(8004, c));
    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);
    cout << "\nPrinting table after inserting 3 new keys and 1 removal:"
<< endl;
    print(lt);

    cout << "\nLet's look up some strings:" << endl;
    try_to_find(lt, 8001);
    try_to_find(lt, 8000);

    cout << "\nTesting the iterator:" << endl;
    LookupTable<int, Mystring>::Iterator it = lt.begin();
    cout << "The first node contains: " << *it << endl;
    while (!it) {
        cout << ++it << endl;
    }
    assert(!lt.cursor_ok());
}

```

```

    lt.go_to_first();
    lt.step_fwd();
    LookupTable<int, Mystring> clt(lt);
    assert(clt.size() == 2);
    assert(strcmp(clt.cursor_datum().c_str(), "I am an ENSF 480 student.")
== 0);
    cout << "\nTest copying: keys should be 8001 and 8002" << endl;
    print(clt);
    lt.remove(8002);
    assert(clt.size() == 2);

    clt = lt;
    assert(clt.size() == 1);
    cout << "\nTest assignment operator: key should be 8001" << endl;
    print(clt);

    lt.make_empty();
    assert(lt.size() == 0);
    cout << "\nPrinting table for the last time: table should be empty" <<
endl;
    print(lt);
    cout << "*** Finished tests on LookupTable<int, Mystring> ***" <<
endl;
}

void test_integer() {
    cout << "\nCreating and testing LookupTable<int, int>..." << endl;
    LookupTable<int, int> lt;

    lt.insert(Pair<int, int>(8002, 9999));
    lt.insert(Pair<int, int>(8001, 8888));
    lt.insert(Pair<int, int>(8004, 8888));
    assert(lt.size() == 3);
    lt.remove(8004);
    assert(lt.size() == 2);
    cout << "\nPrinting table after inserting 3 new keys and 1 removal:"
<< endl;
    print(lt);

```

```

    cout << "\nLet's look up some integers:" << endl;
    try_to_find(lt, 8001);
    try_to_find(lt, 8000);

    cout << "\nTesting the iterator:" << endl;
    LookupTable<int, int>::Iterator it = lt.begin();
    while (!it) {
        cout << ++it << endl;
    }
    assert(!lt.cursor_ok());

    lt.go_to_first();
    lt.step_fwd();
    LookupTable<int, int> clt(lt);
    assert(clt.size() == 2);
    assert(clt.cursor_datum() == 9999);
    cout << "\nTest copying: keys should be 8001 and 8002" << endl;
    print(clt);
    lt.remove(8002);
    assert(clt.size() == 2);

    clt = lt;
    assert(clt.size() == 1);
    cout << "\nTest assignment operator: key should be 8001" << endl;
    print(clt);

    lt.make_empty();
    assert(lt.size() == 0);
    cout << "\nPrinting table for the last time: table should be empty" <<
endl;
    print(lt);
    cout << "*** Finished tests on LookupTable<int, int> ***" << endl;
}

```

FILE: customer.h

```
/*
 * File Name: customer.h
 * Assignment: Lab 3 Exercise C
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

// ENSF 480 - Lab 3, Ex C
// M. Moussavi
#ifndef CUSTOMER
#define CUSTOMER
#include <iostream>

#include "mystring2.h"

using namespace std;

class Customer{
    friend ostream& operator << (ostream&, const Customer& c);

public:
    Customer(const Mystring& fn, const Mystring& ln, const Mystring& ad,
const Mystring& ph);
    const char* getFname() const;
    const char* getLname()const;
    const char* getAddress()const;
    const char* getPhone()const;
    void setFname(const char*s);
    void setLname(const char*s);
    void setAddress(const char*s);
    void setPhone(const char*s);

private:
    Mystring fname;
```

```
    Mystring lname;  
    Mystring address;  
    Mystring phone;  
};  
  
#endif
```

FILE: customer.cpp

```
/*  
 * File Name: customer.cpp  
 * Assignment: Lab 3 Exercise C  
 * Completed By:  
 *   - Sarvesh Vettrivelan  
 *   - Aayush Karthikeyan  
 * Submission Date: Sept. 25, 2026  
 */  
  
// ENSF 480 - Lab 3, Ex C  
// M. Moussavi  
  
#include "customer.h"  
#include <iostream>  
  
using namespace std;  
  
Customer::Customer(const Mystring& fn, const Mystring& ln, const Mystring&  
ad, const Mystring& ph):fname(fn), lname(ln), address(ad), phone(ph)  
{  
}  
  
const char* Customer::getFname() const  
{  
    return fname.c_str();  
}  
  
const char* Customer::getLname() const  
{  
    return lname.c_str();  
}
```

```

}

const char* Customer::getAddress() const
{
    return address.c_str();
}

const char* Customer::getPhone() const
{
    return phone.c_str();
}

void Customer::setFname(const char*s)
{
    Mystring tmp(s);
    fname=tmp;
}

void Customer::setLname(const char*s)
{
    Mystring tmp(s);
    lname=tmp;
}

void Customer::setAddress(const char*s)
{
    Mystring tmp(s);
    address=tmp;
}

void Customer::setPhone(const char*s)
{
    Mystring tmp(s);
    phone=tmp;
}

ostream& operator << (ostream& os, const Customer& c)
{
    os<< "Name: "<< c.getFname();

```

```

os<<" " <<c.getLname();
os<< ". Address: " << c.getAddress();
os<< ". Phone: " << c.getPhone();
return os;
}

```

FILE: mystring2.h

```

/*
 * File Name: mystring2.h
 * Assignment: Lab 3 Exercise C
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#ifndef MYSTRING_H
#define MYSTRING_H
#include <iostream>

class Mystring {
    friend std::ostream& operator<<(std::ostream& os, const Mystring& s);
private:
    int lengthM;
    char* charsM;
public:
    Mystring();
    explicit Mystring(int n);
    Mystring(const char* s);
    Mystring(const Mystring& source);
    ~Mystring();
    Mystring& operator=(const Mystring& rhs);
    int length() const;
    char get_char(int pos) const;
    const char* c_str() const;
    void set_char(int pos, char c);
    Mystring& append(const Mystring& other);
    void set_str(const char* s);
    bool operator<(const Mystring& rhs) const;

```

```

    bool operator>(const Mystring& rhs) const;
    bool operator==(const Mystring& rhs) const;
    bool operator!=(const Mystring& rhs) const;
    bool operator<=(const Mystring& rhs) const;
    bool operator>=(const Mystring& rhs) const;
};
#endif

```

FILE: mystring2.cpp

```

/*
 * File Name: mystring2.cpp
 * Assignment: Lab 3 Exercise C
 * Completed By:
 *   - Sarvesh Vettrivelan
 *   - Aayush Karthikeyan
 * Submission Date: Sept. 25, 2026
 */

#include "mystring2.h"
#include <cassert>
#include <cstring>

Mystring::Mystring() : lengthM(0), charsM(new char[1]) {
    charsM[0] = '\0';
}

Mystring::Mystring(int n) : lengthM(0), charsM(nullptr) {
    assert(n > 0);
    charsM = new char[n];
    charsM[0] = '\0';
}

Mystring::Mystring(const char* s) : lengthM(0), charsM(nullptr) {
    assert(s != nullptr);
    lengthM = static_cast<int>(std::strlen(s));
    charsM = new char[lengthM + 1];
    std::strcpy(charsM, s);
}

Mystring::Mystring(const Mystring& source)
    : lengthM(source.lengthM), charsM(new char[source.lengthM + 1]) {
    std::strcpy(charsM, source.charsM);
}

```

```

}
Mystring::~Mystring() { delete[] charsM; }
Mystring& Mystring::operator=(const Mystring& rhs) {
    if (this != &rhs)
        set_str(rhs.charsM);
    return *this;
}
int Mystring::length() const { return lengthM; }
char Mystring::get_char(int pos) const {
    assert(pos >= 0 && pos < lengthM);
    return charsM[pos];
}
const char* Mystring::c_str() const { return charsM; }
void Mystring::set_char(int pos, char c) {
    assert(pos >= 0 && pos < lengthM && c != '\\0');
    charsM[pos] = c;
}
Mystring& Mystring::append(const Mystring& other) {
    int newLength = lengthM + other.lengthM;
    char* tmp = new char[newLength + 1];
    std::strcpy(tmp, charsM);
    std::strcat(tmp, other.charsM);
    delete[] charsM;
    charsM = tmp;
    lengthM = newLength;
    return *this;
}
void Mystring::set_str(const char* s) {
    assert(s != nullptr);
    int newLength = static_cast<int>(std::strlen(s));
    // Copy before deleting so s may refer to part of this string.
    char* tmp = new char[newLength + 1];
    std::strcpy(tmp, s);
    delete[] charsM;
    charsM = tmp;
    lengthM = newLength;
}
bool Mystring::operator<(const Mystring& rhs) const {
    return std::strcmp(charsM, rhs.charsM) < 0;
}

```

```

bool Mystring::operator>(const Mystring& rhs) const { return rhs < *this;
}
bool Mystring::operator==(const Mystring& rhs) const {
    return std::strcmp(charsM, rhs.charsM) == 0;
}
bool Mystring::operator!=(const Mystring& rhs) const { return !(*this ==
rhs); }
bool Mystring::operator<=(const Mystring& rhs) const { return !(rhs <
*this); }
bool Mystring::operator>=(const Mystring& rhs) const { return !(*this <
rhs); }
std::ostream& operator<<(std::ostream& os, const Mystring& s) {
    return os << s.charsM;
}

```

Exercise C: Program Output

Creating and testing LookupTable<int, Customer>...

Printing table after inserting 3 new keys and 1 removal:

```

8001  Name: Jack Lewis. Address: 12 St. Calgary. Phone:
(403)-1111-123334
8002  Name: Joe Morrison. Address: 11 St. Calgary. Phone:
(403)-1111-123333

```

Let's look up some names:

```

Found key: 8001  Name: Jack Lewis. Address: 12 St. Calgary. Phone:
(403)-1111-123334

```

Sorry, I couldn't find key: 8000 in the table.

Testing the iterator:

```

The first node contains: Name: Jack Lewis. Address: 12 St. Calgary.
Phone: (403)-1111-123334
Name: Jack Lewis. Address: 12 St. Calgary. Phone: (403)-1111-123334
Name: Joe Morrison. Address: 11 St. Calgary. Phone: (403)-1111-123333

```

Test copying: keys should be 8001 and 8002

```

8001  Name: Jack Lewis. Address: 12 St. Calgary. Phone:
(403)-1111-123334
8002  Name: Joe Morrison. Address: 11 St. Calgary. Phone:
(403)-1111-123333

```

Test assignment operator: key should be 8001

```

8001  Name: Jack Lewis. Address: 12 St. Calgary. Phone:
(403)-1111-123334

```

Printing table for the last time: table should be empty

Table is EMPTY.

*** Finished tests on LookupTable<int, Customer> ***

Creating and testing LookupTable<int, Mystring>...

Printing table after inserting 3 new keys and 1 removal:

8001 C++ is a powerful language for engineers but it's not easy.

8002 I am an ENSF 480 student.

Let's look up some strings:

Found key: 8001 C++ is a powerful language for engineers but it's not easy.

Sorry, I couldn't find key: 8000 in the table.

Testing the iterator:

The first node contains: C++ is a powerful language for engineers but it's not easy.

C++ is a powerful language for engineers but it's not easy.

I am an ENSF 480 student.

Test copying: keys should be 8001 and 8002

8001 C++ is a powerful language for engineers but it's not easy.

8002 I am an ENSF 480 student.

Test assignment operator: key should be 8001

8001 C++ is a powerful language for engineers but it's not easy.

Printing table for the last time: table should be empty

Table is EMPTY.

*** Finished tests on LookupTable<int, Mystring> ***

Creating and testing LookupTable<int, int>...

Printing table after inserting 3 new keys and 1 removal:

8001 8888

8002 9999

Let's look up some integers:

Found key: 8001 8888

Sorry, I couldn't find key: 8000 in the table.

Testing the iterator:

8888

9999

Test copying: keys should be 8001 and 8002

8001 8888

8002 9999

Test assignment operator: key should be 8001

8001 8888

```
Printing table for the last time: table should be empty
  Table is EMPTY.
*** Finished tests on LookupTable<int, int> ***

Program terminated successfully.
```